

CTM Workshop for Quarto

Lund 2026

Noah Wolfs-Gäer & Michael Kallweit

19.03.2026

Ruhr University Bochum

Introduction

- First of all, thank you for inviting us to Lund.
- Today, we would like to present Quarto to you.
- (Most of you probably already know it.)
- For those who are new to Quarto, this presentation features a beginner's guide.

Table of contents

For Beginners

Quarto Book

Quarto Extensions

Pyodide-Python

AI-Feedback within Pyodide Python

Quarto Quizzes

Shinylive

Multilanguage Website Projects

Conclusion

For Beginners

What is Quarto?

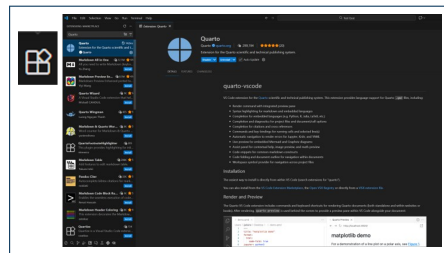
- Website: <https://quarto.org/>
- You write standard Markdown code exactly how you want it to appear.
- You can also embed code (Python, R, Julia, Observable, ...).
- During rendering, Quarto scans the file for code blocks. Each code block is processed by the appropriate engine, and the output is inserted into the document.
- Quarto then uses Pandoc to format the page using Bootstrap themes.
- Finally, it generates an HTML web page, or if you want other formats such as PDF, .docx, ...

How we use it

- We currently use it to demonstrate how Python can illustrate basic mathematics (like curve analysis) or help visualize matrix transformations.
- For this, we create website projects that include “pyodide-python”, a web-based Python environment allowing users to run code without local installation.
- Michael and others expanded a pyodide-python extension to include AI feedback. Michael can probably explain the technical details.
- Quarto can also be used for interactive learning sessions, including quizzes.

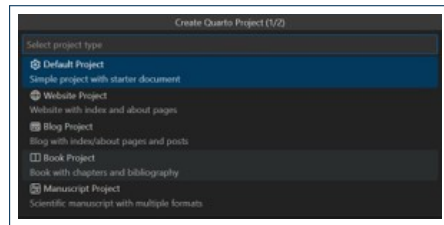
How to get started with Quarto

- Install an IDE of your choice (any text editor works).
- For this guide, we use Visual Studio Code.
- Install VSCode: <https://code.visualstudio.com/download>
- Install Quarto:
<https://quarto.org/docs/get-started/>
- Then, install the Quarto extension for VSCode.
- Click the icon in the bottom left to find extensions.



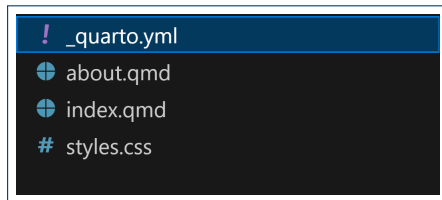
Getting to know Quarto (in VSCode)

- Go to File → New File
- When creating a new Quarto project, the following prompt should appear:
- Here, we select 'Website Project', as that fits our needs.



Getting to know Quarto (in VSCode)

- You should now have 4 files: `_quarto.yml`, `about.qmd`, `index.qmd`, `styles.css`
- These files form the foundation of a Quarto website. You can already render and preview the site (via the terminal with `quarto render` or the render button in the top right of any `.qmd` file).
- The `_quarto.yml` is the configuration file for the website; here you can adjust the core design and global settings.
- Each `.qmd` file has a YAML header (marked by `---` at the top and bottom). Here you can override settings for that specific page.



- `#, ##, ###` creates sections, subsections, subsubsections, etc.
- `:::{} ...:::` creates callout blocks or tabsets, depending on the content inside `{}`.
- `<details> </details>` creates a foldable HTML block. You can add a title using `<summary> </summary>` inside.
- ````{} ```` creates a code block. The content of `{}` defines the language.

Output Formats

- By default, Quarto renders to **HTML**.
- The format is set in the YAML header via `format:`.
- Multiple formats can be rendered at once. Quarto will produce all of them with a single `quarto render`.
- **PDF** requires a $\text{\LaTeX}$ installation. The easiest option is TinyTeX, which Quarto can install for you: `quarto install tinytex`

Single format

```
format: pdf
```

Multiple formats

```
format:  
  html: default  
  pdf: default  
  epub: default  
  docx: default
```

Quarto Book

What is a Quarto Book?

- A Quarto **Book** is a project type designed for long-form, multi-chapter documents.
- Chapters are individual `.qmd` files, ordered in `_quarto.yml` under a `book:` key.
- Key advantage over a plain website: **automatic chapter numbering** and **cross-references across chapters**.
- Output formats: HTML (with sidebar navigation & full-text search), PDF, EPUB, MS Word.
- All rendered files are collected in a `_book/` directory.
- Ideal for course scripts, lecture notes, and open textbooks exactly the kind of material we create for CTM.

Setting up a Quarto Book

Creating a new book project:

- Terminal: `quarto create project book`
- VSCode: Command Palette → *Quarto: Create Project* → *Book Project*
- Render with `quarto render` or preview live with `quarto preview`.

Typical `_quarto.yml`:

- Set `project: type: book`
- List chapters in order under `book: chapters:`

Example `_quarto.yml`

```
project:
  type: book
book:
  title: "CTM Script"
  author: "RUB CTM"
  chapters:
    - index.qmd
    - chapter1.qmd
    - chapter2.qmd
```

Quarto Extensions

What are Quarto Extensions?

- Quarto extensions are community-contributed add-ons that enhance Quarto's functionality.
- They can add new **shortcodes**, **filters**, **formats**, or **journal templates**.
- Extensions are hosted on GitHub and managed directly through the Quarto CLI.
- The official listing of extensions can be found at:
<https://quarto.org/docs/extensions/>

Installing and Using Extensions

Installing an extension:

- Run `quarto add user/repo` in your project directory.
- Example: `quarto add quarto-ext/fontawesome`
- The extension is stored locally in `_extensions/`.

Using an extension:

- **Filters:** add `filters: [extension-name]` to the YAML header.
- **Shortcodes:** use `{{< shortcode >}}` inline in your document.
- **Formats:** set `format: extension-name-html` in the YAML header.

Note

Extensions are project-local. You install them once per project and commit `_extensions/` to version control so collaborators don't need to reinstall.

Notable Quarto Extensions

Interactive & Code

- **pyodide** run Python in the browser (WebAssembly)
- **webr** run R in the browser (WebAssembly)
- **shinylive** embed Shiny apps without a server

Learning & Teaching

- **quizdown** multiple-choice and ordering quizzes
- **countdown** in-slide countdown timer

Design & Layout

- **fontawesome** `{{< fa icon >}}` shortcode for icons
- **lightbox** click-to-zoom image galleries
- **roughnotation** hand-drawn style annotations
- **animate** CSS scroll animations

Publishing

- **academicons** academic/research icons
- Journal templates (JASA, Elsevier, ...)

Pyodide-Python

- Pyodide-Python is a web-based Python environment that allows you to run Python without local installation, as all computing is done in the browser.
- This is especially useful for giving students an easy entry into using Python for **Computational Thinking in Mathematics**.
- The extension can be installed via the terminal with
`quarto add coatless-quarto/pyodide`.
- The extension is open-source and can be found at:
<https://github.com/coatless-quarto/pyodide>

Pyodide-Python

- To use the extension in your .qmd files, you need to add `filters: - pyodide` to the YAML header.
- Afterward, you can use ````{pyodide-python} ...```` to embed Python code blocks into the page.

- For example:

```
```{pyodide-python} print("Hello World") ```
```

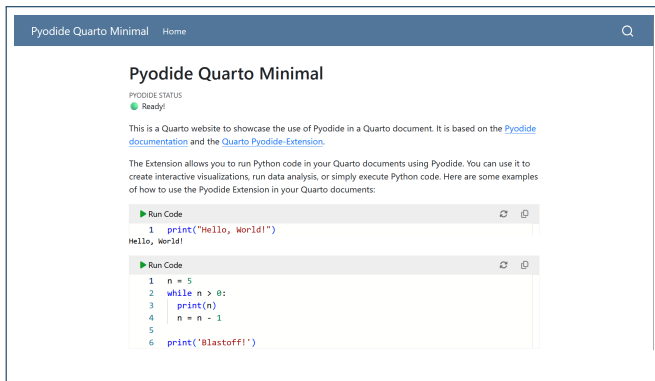
will create a code block containing

```
print("Hello World")
```

, which can be edited and executed on the website.

```
1 ---
2 title: "Pyodide Quarto Minimal"
3 filters:
4 - pyodide
5 ---
```

# Pyodide-Python in Action



A minimal example of a website using the Pyodide-Extension can be found in this repository or directly at: [https://gitlab.ruhr-uni-bochum.de/ctm/lund-meeting/-/tree/main/Pyodide%20Quarto%20Minimal?ref\\_type=heads](https://gitlab.ruhr-uni-bochum.de/ctm/lund-meeting/-/tree/main/Pyodide%20Quarto%20Minimal?ref_type=heads)

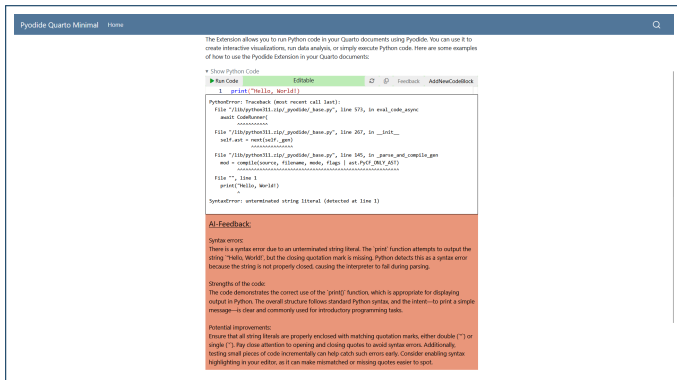
# AI-Feedback within Pyodide Python

---

- At Ruhr University Bochum, we modified the **coatless-pyodide** extension to include AI feedback.
- The extension works the same as the original but adds the ability to ask an AI for help with the code.
- To use this feature, users must provide a valid **API Key** and **Base URL**.



# Pyodide-Python with AI-Feedback in Action



The screenshot shows a web interface for 'Pyodide Quarto Minimal'. At the top, there's a navigation bar with 'Pyodide Quarto Minimal' and 'Home' links, and a search icon. Below the navigation bar, a text block explains that the extension allows running Python code in Quarto documents using Pyodide. A 'Show Python Code' button is visible. The main area contains a code editor with the following Python code:

```
1 print("Hello, World!")
```

Below the code editor, a 'PythonError' traceback is displayed, showing a 'SyntaxError: unterminated string literal (detected at line 1)'. This is followed by an 'AI-Feedback' section with a red background, which provides a detailed explanation of the syntax error and offers suggestions for improvement.

**AI-Feedback:**

**Syntax errors:**

There is a syntax error due to an unterminated string literal. The 'print' function attempts to output the string "Hello, World!", but the closing quotation mark is missing. Python detects this as a syntax error because the string is not properly closed, causing the interpreter to fail during parsing.

**Strengths of the code:**

The code demonstrates the correct use of the 'print()' function, which is appropriate for displaying output in Python. The overall structure follows standard Python syntax, and the intent—to print a simple message—is clear and commonly used for introductory programming tasks.

**Potential improvements:**

Ensure that all string literals are properly enclosed with matching quotation marks, either double (") or single ('). Pay close attention to opening and closing quotes to avoid syntax errors. Additionally, testing small pieces of code incrementally can help catch such errors early. Consider enabling syntax highlighting in your editor, as it can make mismatched or missing quotes easier to spot.

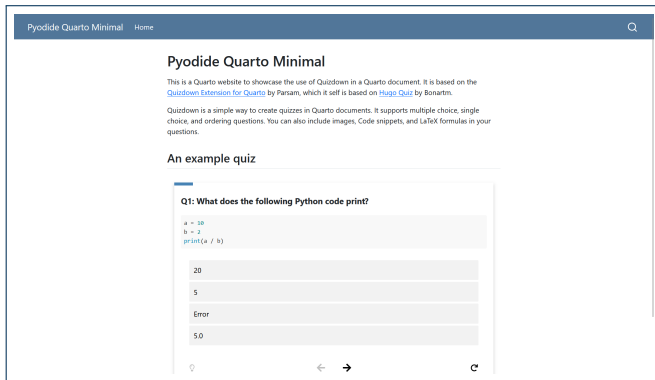
A minimal example of a website using the modified Pyodide-Extension can be found in this repository or directly at: [https://gitlab.ruhr-uni-bochum.de/ctm/lund-meeting/-/tree/main/Pyodide-Feedback%20Minimal?ref\\_type=heads](https://gitlab.ruhr-uni-bochum.de/ctm/lund-meeting/-/tree/main/Pyodide-Feedback%20Minimal?ref_type=heads)

## Quarto Quizzes

---

- Gamification can greatly enhance learning, so we wanted to provide a gamified experience in our tutorials.
- For this purpose, we looked and found the **Quizdown** extension:  
`https://github.com/parmsam/quarto-quizdown`
- Quizdown allows you to create single-choice, multiple-choice, and ordering quizzes without much extra effort.

# Quizdown in Action

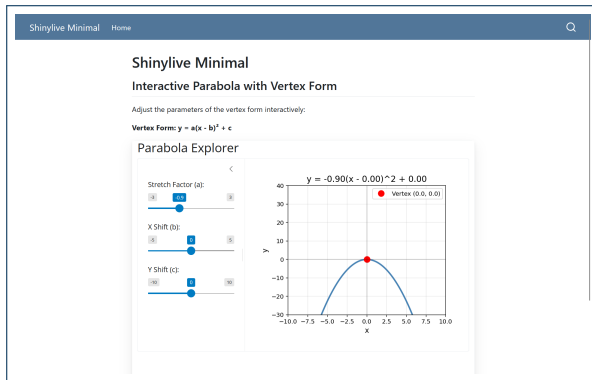


A minimal example of a website with Quizdown can be found in this repository or directly at:  
[https://gitlab.ruhr-uni-bochum.de/ctm/lund-meeting/-/tree/main/Quizdown%20Minimal?  
ref\\_type=heads](https://gitlab.ruhr-uni-bochum.de/ctm/lund-meeting/-/tree/main/Quizdown%20Minimal?ref_type=heads)

**Shinylive**

---

- In further enhancing the interactive experience we looked for something that would allow us to give users direct feedback on mathematical concepts.
- For that we looked into Shiny, specifically the shinylive extension for Quarto:  
`https://github.com/quarto-ext/shinylive`
- Shinylive is an extension that lets you embed Shiny apps into your Quarto website.
- The app runs entirely in the browser (via WebAssembly), and the code can be edited live with sliders, making the consequences immediately visible.



A minimal example of a website using the shinylive Extension can be found in this repository or directly at: [https://gitlab.ruhr-uni-bochum.de/ctm/lund-meeting/-/tree/main/Shinylive%20Minimal?ref\\_type=heads](https://gitlab.ruhr-uni-bochum.de/ctm/lund-meeting/-/tree/main/Shinylive%20Minimal?ref_type=heads)

# Multilanguage Website Projects

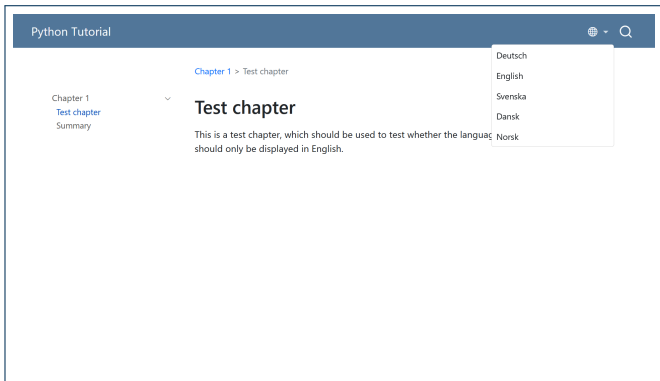
---



# Multilanguage Website Projects

- As this is an international project, we wanted to give users the ability to switch between languages in Quarto.
- Currently, there is no official feature to switch languages within a specific part of the website; it only supports redirecting to a different homepage.
- Therefore, we wrote a small Python script that modifies the navbar to include a dropdown menu, allowing users to switch languages while staying on the same page.
- Quarto's `preview` command does not support the switching between languages, so you won't be able to properly preview your page using `quarto preview`
- Instead, run the included Python script to host the site locally and preview it in the browser.

# Multilanguage Website Projects in Action



A minimal example of a website with multi-language support (I hope the AI translations aren't too bad!) can be found in this repository or directly at: [https://gitlab.ruhr-uni-bochum.de/ctm/lund-meeting/-/tree/main/Multilanguage%20Website%20Setup?ref\\_type=heads](https://gitlab.ruhr-uni-bochum.de/ctm/lund-meeting/-/tree/main/Multilanguage%20Website%20Setup?ref_type=heads)

## Conclusion

---

# Conclusion

- Quarto is a powerful tool for creating interactive and engaging educational content, especially in the context of Computational Thinking in Mathematics.
- There is a vast number of extensions available, providing many possibilities to enhance the student experience.
- As some of you have already seen, Quarto is a user-friendly tool for creating scripts, as it allows you to combine code and output in a single file.

All materials from this presentation and the slides can be found at:

<https://gitlab.ruhr-uni-bochum.de/ctm/lund-meeting>